

# LESO 1033 MRAP Training Plan PDF — Dynamic Extraction Pipeline

Fully automated pipeline: PDF → pdf2image (page images) → pytesseract (OCR) → page grouping (boundary detection) → regex field extraction → CSV output

No hardcoded records. Every row is derived from live OCR output.

Usage: python leso\_extraction\_pipeline.py [path/to/file.pdf]

Dependencies:

- pip install pdf2image pytesseract
- apt-get install tesseract-ocr poppler-utils

```
In [1]: import csv
import re
import sys
import textwrap
from pathlib import Path

from pdf2image import convert_from_path
import pytesseract
```

```
In [2]: # CONFIG

PDF_PATH      = "/Users/jennifersaucedo/Desktop/MS Engineering Data Science/Summer 2026/LESO 1033 MRAP Training Plan.pdf"
OUTPUT_PATH    = "/Users/jennifersaucedo/Desktop/MS Engineering Data Science/Summer 2026/LESO 1033 MRAP Training Plan.csv"
OCR_DPI        = 200    # higher = better quality, slower
SUMMARY_LEN    = 300    # max chars for Summary field

FIELDNAMES = [
    "Report_ID", "Incident_Type", "Date", "Location",
    "Agency", "Officer", "Summary", "Suspect_Description",
    "Outcome", "Source_Pages",
]
```

```
In [3]: # Check where you're actually running this
import os
print(os.getcwd())          # current working directory
print(os.path.exists(PDF_PATH)) # quick sanity check
```

/Users/jennifersaucedo/Desktop/MS Engineering Data Science/Summer 2026/LESO 1033 MRAP Training Plan  
True

```
In [4]: # STEP 1 – OCR: convert every PDF page to text

def ocr_pdf(PDF_PATH: str, dpi: int = OCR_DPI) -> dict[int, str]:
    """
    Convert each PDF page to an image and run Tesseract OCR.
```

```

Returns {page_number: raw_text} (1-indexed).
"""
print(f"[OCR] Converting PDF to images at {dpi} DPI ...")
images = convert_from_path(PDF_PATH, dpi=dpi)
print(f"[OCR] Running Tesseract on {len(images)} pages ...")
pages = {}
for i, img in enumerate(images, start=1):
    pages[i] = pytesseract.image_to_string(img)
    if i % 10 == 0:
        print(f"    ... {i}/{len(images)} pages done")
print(f"[OCR] Complete - {len(pages)} pages extracted")
return pages

```

In [5]: *# STEP 2 – GROUP pages into logical documents*

```

# Patterns that reliably signal the start of a new document
_NEW_DOC_SIGNALS = re.compile(
    r'To Whom It May Concern'
    r'|RE\s*:'
    r'|Dear\s+\w+'
    r'|PURPOSE\s*:'
    r'|POLICIES AND PROCEDURES'
    r'|COURSE\s*:'
    r'|LESSON TITLE\s*:'
    r'|Lesson Plan'
    r'|Standard Operating Procedure'
    r'|MEMORANDUM'
    r'|LAW ENFORCEMENT AGENCY.*REQUEST'
    r'|INVOICE',
    re.IGNORECASE,
)

def group_pages(pages: dict[int, str]) -> list[dict]:
    """
    Walk pages in order. When a new-document signal is found, close the
    current group and start a new one.
    Returns list of {page_nums, text} dicts.
    """
    groups = []
    current_pages = []
    current_text = []

    for page_num in sorted(pages):
        text = pages[page_num]
        is_new = bool(_NEW_DOC_SIGNALS.search(text))

        if is_new and current_pages:
            groups.append({
                "page_nums": current_pages[:],
                "text": "\n".join(current_text),
            })
            current_pages = []
            current_text = []

        current_pages.append(page_num)

```

```

        current_text.append(text)

    # flush final group
    if current_pages:
        groups.append({
            "page_nums": current_pages,
            "text": "\n".join(current_text),
        })

    print(f"[GROUP] {len(pages)} pages → {len(groups)} logical documents")
    return groups

```

In [6]: # STEP 3 – FIELD EXTRACTION via regex

```

# — Date —
_DATE_RE = re.compile(
    r'\b(?:January|February|March|April|May|June|July|August|'
    r'September|October|November|December)\s+\d{1,2},?\s+\d{4}'
    r'|\b\d{1,2}\s+(?:January|February|March|April|May|June|July|August|'
    r'September|October|November|December)\s+\d{4}'
    r'|\b\d{1,2}/\d{1,2}/\d{2,4}\b',
    re.IGNORECASE,
)

# — Agency —
_AGENCY_RE = re.compile(
    r'\b([A-Z][A-Za-z ]+)?'
    r'(?:Sheriff\s?(?:\s+(?:Office|Department)|Police\s+Department|'
    r'Police\s+Dept|Sheriff\s+Department))\b',
)

# — Location —
_LOCATION_RE = re.compile(
    r'([A-Z][a-zA-Z ]+, \s*(?:Arkansas|AR)\s*\d{0,5})',
)

# — Officer / Signer —
_OFFICER_RE = re.compile(
    r'\b(?:Sheriff|Chief|Lt\.|Lieutenant|Cpl\.|Corporal|Capt\.|Captain|'
    r'Deputy|Major|Sgt\.|Sergeant|Officer|Ofc\.|Commander)'
    r'\s+[A-Z][a-z]+(?:\s+[A-Z]\.)(?:\s+[A-Z][a-z]+){0,2}',
)

# — Incident / Document type —
_TYPE_HINTS = [
    (re.compile(r'INVOICE', re.I), "Invoice"),
    (re.compile(r'MEMORANDUM', re.I), "Internal Memorandum"),
    (re.compile(r'LAW ENFORCEMENT AGENCY.*REQUEST', re.I), "LEA Equipment Re"),
    (re.compile(r'LESSON TITLE|Lesson Plan', re.I), "Training Lesson Plan"),
    (re.compile(r'POLICIES AND PROCEDURES', re.I), "Policies and Procedures"),
    (re.compile(r'Standard Operating Procedure|SOP', re.I), "Standard Operat"),
    (re.compile(r'intended use and training', re.I), "LESO Intended Use Decl"),
    (re.compile(r'capability|training.*program', re.I), "Vendor Capability S"),
    (re.compile(r'certificate.*awarded|awarded.*certificate', re.I), "Traini"),
    (re.compile(r'RE\s*:\s*MRAP|RE\s*:\s*Mine Resistant', re.I), "MRAP Polic"),
    (re.compile(r'aviation unit|helicopter', re.I), "Aviation Unit Policy")

```

```

]

def classify_incident_type(text: str) -> str:
    for pattern, label in _TYPE_HINTS:
        if pattern.search(text):
            return label
    return "Policy/Correspondence"

# — Subject line —
_SUBJECT_RE = re.compile(
    r'(?:(?:RE\s*|Ref\s*|Subject\s*|RE\s*\n)\s*(.+))', re.IGNORECASE
)

# — Outcome keywords —
_OUTCOME_HINTS = [
    (re.compile(r'approved', re.I), "Approved"),
    (re.compile(r'certificate.*award', re.I), "Certificate Issued"),
    (re.compile(r'policy.*implement|implement.*policy', re.I), "Policy Imple"),
    (re.compile(r'training.*complet', re.I), "Training Completed"),
    (re.compile(r'request.*submit|submit.*request', re.I), "Request Submitte"),
    (re.compile(r'SOP.*enact|enact.*SOP', re.I), "SOP Enacted"),
    (re.compile(r'declaration.*filed|filed.*declaration', re.I), "LESO Decla")
]

def extract_outcome(text: str) -> str:
    for pattern, label in _OUTCOME_HINTS:
        if pattern.search(text):
            return label
    # Fall back: last meaningful sentence
    sentences = [s.strip() for s in re.split(r'[.!?]', text) if len(s.strip()) > 0]
    return sentences[-1][:120] if sentences else "See summary"

def extract_fields(group: dict) -> dict:
    """Extract all CSV fields from a grouped document's combined OCR text."""
    text = group["text"]
    pages = group["page_nums"]

    # Date — take first found
    dates = _DATE_RE.findall(text)
    date = dates[0].strip() if dates else ""

    # Agency — most frequent cleaned match
    raw_agencies = _AGENCY_RE.findall(text)
    cleaned = [re.sub(r'\s+', ' ', a).strip() for a in raw_agencies]
    agency = max(set(cleaned), key=cleaned.count) if cleaned else ""

    # Location — first city, AR match
    locs = _LOCATION_RE.findall(text)
    location = locs[0].strip() if locs else ""
    location = re.sub(r'\s+', ' ', location).strip()

    # Officer — first ranked name found
    officers = _OFFICER_RE.findall(text)
    officer = re.sub(r'\s+', ' ', officers[0]).strip() if officers else ""

```

```

# Incident type
incident_type = classify_incident_type(text)

# Summary – meaningful lines, trimmed to SUMMARY_LEN chars
lines = [l.strip() for l in text.splitlines() if len(l.strip()) > 40]
summary = " | ".join(lines[:5])
summary = re.sub(r'\s+', ' ', summary)[:SUMMARY_LEN]

# Outcome
outcome = extract_outcome(text)

# Source pages
if len(pages) == 1:
    source_pages = str(pages[0])
else:
    source_pages = f"{pages[0]}–{pages[-1]}"

return {
    "Incident_Type": incident_type,
    "Date": date,
    "Location": location,
    "Agency": agency,
    "Officer": officer,
    "Summary": summary,
    "Suspect_Description": "N/A",
    "Outcome": outcome,
    "Source_Pages": source_pages,
}

```

In [7]: # STEP 4 – ASSEMBLE RECORDS

```

def build_records(groups: list[dict]) -> list[dict]:
    records = []
    for i, group in enumerate(groups, start=1):
        fields = extract_fields(group)
        fields["Report_ID"] = f"RPT_{i:03d}"
        # Reorder to match FIELDNAMES
        records.append({col: fields.get(col, "") for col in FIELDNAMES})
    return records

```

In [8]: # STEP 5 – WRITE CSV

```

def write_csv(records: list[dict], path: str) -> None:
    Path(path).parent.mkdir(parents=True, exist_ok=True)
    with open(path, "w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=FIELDNAMES)
        writer.writeheader()
        writer.writerows(records)
    print(f"[CSV] Written -> {path} ({len(records)} rows, {len(FIELDNAMES)})

```

In [9]: # STEP 6 – PRINT SUMMARY

```

def print_summary(records: list[dict]) -> None:
    from collections import Counter
    print("\n" + "=" * 60)

```

```

print("  EXTRACTION SUMMARY")
print("=" * 60)
print(f"  Records extracted : {len(records)}")
agencies = [r["Agency"] for r in records if r["Agency"]]
print(f"  Unique agencies   : {len(set(agencies))}")

types = Counter(r["Incident_Type"] for r in records)
print("\n  Document types:")
for t, n in types.most_common():
    print(f"    {n:>2}x {t}")

years = Counter(re.search(r'\d{4}', r["Date"]).group()
                for r in records if re.search(r'\d{4}', r["Date"]))
print("\n  Records by year:")
for yr, n in sorted(years.items()):
    print(f"    {yr}: {'█' * n} ({n})")

print("\n  Sample records:")
for r in records[:3]:
    print(f"\n    [{r['Report_ID']}] {r['Incident_Type']}")
    print(f"      Date       : {r['Date']}")
    print(f"      Agency     : {r['Agency']}")
    print(f"      Officer    : {r['Officer']}")
    print(f"      Pages      : {r['Source_Pages']}")
    print(f"      Summary    : {r['Summary'][:100]}...")
print("=" * 60)

```

In [10]: # MAIN

```

if __name__ == "__main__":
    # pdf_path = sys.argv[1] if len(sys.argv) > 1 else PDF_PATH

    print("=" * 60)
    print("  LESO 1033 MRAP – DYNAMIC EXTRACTION PIPELINE")
    print("=" * 60)
    print(f"  Input   : {PDF_PATH}")
    print(f"  Output  : {OUTPUT_PATH}")

    # Step 1 – OCR
    pages = ocr_pdf(PDF_PATH)

    # Step 2 – Group into logical documents
    groups = group_pages(pages)

    # Step 3+4 – Extract fields and assemble records
    print("[EXTRACT] Running field extraction on each document group ...")
    records = build_records(groups)

    # Step 5 – Write CSV
    write_csv(records, OUTPUT_PATH)

    # Step 6 – Summary
    print_summary(records)

```

```

=====
LESO 1033 MRAP – DYNAMIC EXTRACTION PIPELINE
=====
Input  : /Users/jennifersaucedo/Desktop/MS Engineering Data Science/Summer
2026/LES02.pdf
Output : /Users/jennifersaucedo/Desktop/MS Engineering Data Science/Summer
2026/leso_mrap_structured_output.csv
[OCR] Converting PDF to images at 200 DPI ...
[OCR] Running Tesseract on 75 pages ...
    ... 10/75 pages done
    ... 20/75 pages done
    ... 30/75 pages done
    ... 40/75 pages done
    ... 50/75 pages done
    ... 60/75 pages done
    ... 70/75 pages done
[OCR] Complete – 75 pages extracted
[GROUP] 75 pages → 27 logical documents
[EXTRACT] Running field extraction on each document group ...
[CSV] Written → /Users/jennifersaucedo/Desktop/MS Engineering Data Science/S
ummer 2026/leso_mrap_structured_output.csv (27 rows, 10 cols)

```

---



---

#### EXTRACTION SUMMARY

---



---

Records extracted : 27  
Unique agencies : 17

#### Document types:

- 6× Training Lesson Plan
- 6× Policy/Correspondence
- 4× Vendor Capability Statement
- 3× LESO Intended Use Declaration
- 2× Policies and Procedures
- 1× Standard Operating Procedure
- 1× Internal Memorandum
- 1× Invoice
- 1× LEA Equipment Request
- 1× MRAP Policy Letter
- 1× Training Certificate

#### Records by year:

2014: █████ (4)  
2015: █████ (13)

#### Sample records:

[RPT\_001] Vendor Capability Statement  
Date : May 26, 2015  
Agency : Benton County Sheriff's Office  
Officer : Sheriff Kelley Cradduck Major  
Pages : 1-2  
Summary : Major Nathan Atchison Chief Deputy R.L. Conner Major Shawn Ho  
lloway | Bentonville, Arkansas 72712 Ph...

[RPT\_002] Policies and Procedures

Date : 6/1/15  
Agency : Benton County Sheriff's Office  
Officer :  
Pages : 3-4  
Summary : The Benton County Sheriff's Office obtained a Mine Resistant Ambush Protected (MRAP) | Vehicle throu...

[RPT\_003] Training Lesson Plan  
Date : 4/24/2015  
Agency : Benton Police Department  
Officer : Capt. Scotty L. Hodges  
Pages : 5-8  
Summary : Capt. Scotty L. Hodges, Benton Police Department | This 4 hour block of training will consist of cla...

---